

More on service layers in Django

 [March 23, 2020](#)  [Django](#), [Python](#)

Well, [that](#) provoked some discussion.

While there were plenty of people who agreed with the general idea of that post, there were also quite a few objections. And most of those seem to fall into two main categories: people who want some type of additional layer (and may or may not call it a “service”) as a way of managing cross-cutting complexity, and people who want it as an isolating abstraction for testing.

There’s also a third group whose objections are more about the Active Record ORM pattern, but that’s not something that can be solved within the context of the Django ORM — it is and likely always will be an Active Record ORM. As I mentioned in the previous post, if you want a Data Mapper ORM in Python, I think you really should be using [SQLAlchemy](#). And you probably *shouldn’t* be using Django at all in that case — instead, either go with a microframework that lets you plug in whatever you want (most likely [Flask](#)), or with a full-stack framework built around SQLAlchemy (your best choice is probably [Pyramid](#)). I think you’ll be much happier working with one of those than with trying to retrofit SQLAlchemy, or any other replacement ORM, into Django.

So today I’m going to focus mainly on the other two sets of objections. And if you haven’t read [the previous post](#), I do recommend you take a look at it now for background.

Managing complexity

A lot of this group of objections started out with something like “the examples you mentioned are all basically tiny little toy projects where of course it’s easy to implement the business logic on the models, but I have *real*, really complex applications, so what’s your advice for those?”

And the answer is: I gave my advice for how to implement “business logic” in Django, and I stand by it, for projects of any size. Like I said previously, that advice comes from around 15 years of working with Django, on projects large and small. But the real thing people seemed to be getting at, when prodded a bit more, was how to manage cross-cutting complexity when implementing methods on individual models. An example one commenter brought up: if cancelling a recurring billing agreement has to update a half-dozen other things (sending notifications, cancelling any pending orders/shipments, etc.

etc.), this argument says, it makes no sense to do it all in a method on a `BillingAgreement` model. So you have to have a service layer where you do that!

I agree with the first part of that: it doesn't make sense to do all those things in a method on a single model. But the conclusion — you need a service layer for this — doesn't follow. First of all, hoisting those half-dozen (or potentially more) things out of a single method on a `BillingAgreement` model and into a single method on, say, a `BillingService` doesn't actually solve the real problem, which is that a single method is trying to do too much. Taking that over-complex method and just moving it, as-is, into a different place in your code won't make it stop being over-complex, and won't make it OK for it to be that complex.

I was hesitant to get into this in the previous post because there's no easy answer. There are a lot of patterns for designing software to handle this type of complexity without having to put huge chunks of it into just a few overburdened methods or classes, but none of them are one-size-fits-all. To take one example: in the reddit thread for the previous post I mentioned that several larger codebases (both Django and non-) I've worked with have eventually integrated, or at least bolted on, some type of publish/subscribe message broker. This can be a useful solution, because often what's really wanted when breaking up these complex cross-cutting methods is a way to have many different components work together but not necessarily have to possess knowledge of, or directly invoke, each other. Having each component publish messages about what it's doing, and requiring others to subscribe and react to the specific types of messages they care about, can accomplish this.

But that's far from the only possible solution, and depending on what you're doing it may be entirely wrong, or otherwise have disadvantages that outweigh the advantages. The real answer here is not to memorize one solution and apply it everywhere; it's to build up broader familiarity with patterns and principles for designing complex software systems, and take care in choosing which ones you use in a given situation. Which is certainly easier said than done — there are a lot of documented patterns out there and a huge amount of literature advocating for or against particular ones — but again there just isn't an easy universal solution. If there were, we'd all be able to adopt it and get on with our lives.

And once you start adopting design patterns that let you break up and manage the type of cross-concern complexity so many people brought up, you no longer have, or no longer have so many, complex chunks of code that need to do lots of different things. Which then gets back into implementing “business logic” operations directly on your models (or their auxiliary classes like managers and querysets, as appropriate). This is a

big reason why I put a long aside about the Law of Demeter (also known as the principle of least knowledge — minimizing how many other things and how many details about them any given piece of code needs to know) in the original post. A lot of the motivation behind service layers, or other dedicated places to put “business logic” separate from other concerns, in my experience, comes from the difficulty of adhering to these sorts of deep design principles. I’ve been as guilty of this as anyone over the course of my career; it’s awfully convenient sometimes to bend or even break these principles to get something done and out the door quickly. But the bill for that also always comes due sooner or later.

Isolation and testing

This was the less common objection, but several instances of it came in messages from people I know and respect, and who made more generally thoughtful arguments than some of the other discussions I saw. In its simplest form, this argument may seem like what I already talked about in the previous post: using a layer of indirection to make it easier to swap out the ORM. But the key difference is that where other people will want this to make it easier to permanently remove/replace the data access layer, this group wants to be able to *temporarily* replace the ORM during test runs, but have it present and available at all other times.

This is often part of a more general approach of trying to reduce the number of dependencies involved in running tests (and in this case, both the ORM and the database(s) it will talk to are dependencies). One benefit of this is faster test runs; another benefit is needing less infrastructure to get the tests to run. And there are also philosophical arguments about unit tests versus integration tests which drive people toward wanting the ability to test individual pieces of application logic in a way that doesn’t turn into an implicit test of the full component stack.

I’ll admit I find it more difficult to argue against this than the other reasons people advance for wanting a service layer abstraction in Django, because there’s an extent to which I agree with the underlying idea: I like being able to test specific pieces of logic in isolation from the overall system they’ll be a part of, and I routinely use a variety of techniques to make this easier to do.

But I’m not able to go all the way on this argument, for a couple of reasons.

One is that I don’t really believe in full isolation of “logic”. It may well be I’m just not good enough to pull it off, but I’ve had some downright embarrassing experiences when

attempting it. There are a lot of things that can go wrong when using mocks, fake-factories and other tools to simulate a dependency, and it's easy to wind up with misplaced confidence because the beautifully-isolated tests were operating with incorrect or incomplete simulations of key dependencies, or even just testing against the wrong things. So generally, the more complex or crucial to the application a dependency is, the less likely I am to try to isolate/mock it away and the more likely I am to use the real thing. A hybrid approach of trying to have some "pure" tests that use simulated dependencies and other less-pure tests that use the real thing is possible, of course, but raises questions about why so much effort is put into isolating the "logic" from the "dependencies" if test runs are going to have to use the real dependencies at some point anyway.

And in the specific domain of web applications, I tend to see the database as a sufficiently complex/crucial dependency that I don't think trying to isolate it away in tests is a good approach.

The other reason is that I see re-architecting for testability as a diminishing-returns project. That is, at first the changes you'll make for testability are mostly easy wins and usually produce a net gain in overall code quality, or at worst are neutral in its impact on other metrics. But past a certain point the gains in testability begin to be outweighed by the amount of code churn to accomplish them and by losses in other forms of quality. When it comes to introducing entire new layers of abstraction/indirection in front of major components, I think that diminishing-returns point has often been passed.

And even without the ORM-specific objections from the last post, it's important to remember those layers don't come for free: they become a permanent part of the codebase that you and every other developer on your team will have to work with and account for. Especially as a project grows larger, it's vital to treat complexity as a budgeted resource, and use care in deciding where and how to spend that budget. While I suspect others have different opinions, I'm not convinced that introducing service layers in Django apps will produce enough overall gain to justify the chunk of the complexity budget that has to be spent on it.

A wrap (for now)

At this point I think I'm pretty much argued out on this topic; if you're in the camp of wanting service layers in Django apps and I haven't changed your mind with the roughly 5,000 words of these two posts I doubt I'd be able to do it with 10,000 more. If you were curious or on the fence about whether service layers in Django apps are a good idea, or

just wanting to know what an argument against them would look like, hopefully I've been able to lay out my take on it clearly enough.

Next time: I'm not sure yet what topic, but probably something *other* than service layers in Django apps.